

802.1x Flooding Attack

Marina Schwerz Bon¹, Pedro Boeira Webber¹

¹Universidade do Vale do Rio dos Sinos (UNISINOS)
Porto Alegre – RS – Brazil

Abstract. *This article presents the design, deployment, and security evaluation of an IEEE 802.1X-protected wireless network implemented on a Raspberry Pi platform. Leveraging FreeRADIUS, hostapd, and dnsmasq, a realistic WPA2 environment, which includes RADIUS server setup, 802.1X authenticator configuration, DHCP setup, and network forwarding, was recreated. Building on this groundwork, a persistent deauthentication flooding attack, exploiting the lack of cryptographic protection on management frames to deny service without ever breaking the underlying encryption, is demonstrated. Findings underscore a critical 802.1X vulnerability: unprotected messages enable simple, high-impact denial-of-service attacks in protocol 802.1x.*

1. Introduction

IEEE 802.1X is a standard for port-based network access control that provides an authentication framework for devices trying to connect to a LAN or WLAN. In wireless networks, 802.1X is the foundation of WPA2 security, governing how clients (supplicants) must authenticate (via the Extensible Authentication Protocol – EAP) to a network access point (authenticator), which relays credentials to an authentication server (typically RADIUS). It enables centralised user identification, authentication, and authorisation for Wi-Fi, often using credentials (username/password or certificates) instead of a shared password. This helps enterprises ensure that only authorised users or devices gain network access, thereby mitigating the risks of unauthorised entry. Since its introduction, WPA/WPA2-Enterprise (802.1X with robust EAP methods) has been considered the standard for wireless security, addressing many of the well-known flaws of WEP and WPA-Personal by providing per-session encryption keys and mutual authentication.

However, while 802.1X enhances authentication and key management, it is not immune to vulnerabilities. The protocol's design and interactions with Wi-Fi (IEEE 802.11) have revealed several weaknesses over time. Researchers have demonstrated that poor implementation or lack of protective measures in 802.1X can lead to issues such as session hijacking, man-in-the-middle (MitM) attacks, and denial-of-service (DoS) conditions. In particular, the control and management frames used in the 802.1X authentication process and the wireless management frames (like deauthentication frames) were not originally protected cryptographically, leaving them susceptible to spoofing or flooding by attackers. In the rest of this article, we configure a test wireless environment with 802.1X and demonstrate a flooding attack that exploits weaknesses. The demonstration uses a Raspberry Pi-based 802.1X environment and open-source tools (FreeRADIUS, hostapd, aircrack-ng suite, Wireshark) to illustrate how an attacker can disrupt a wireless network secured with 802.1X by forcing re-authentication and flooding the air with malicious frames.

2. Literature Review

Vulnerabilities in 802.1X [Arbaugh and Mishra 2002a]: Shortly after 802.1X's introduction, an initial security analysis of the standard revealed fundamental vulnerabilities. Arbaugh and Mishra (2002) outlined two ways the standard falls short in providing access control and authentication. First, they demonstrated that an attacker could perform a session hijacking. After a legitimate client successfully authenticates and the port is opened, an attacker could potentially take over the session because 802.1X does not bind the session to the original client's identity. Second, they outlined the possibility of a man-in-the-middle attack during the authentication phase, given the lack of integrity protection on EAP messages and the reliance on higher-layer EAP methods that might be vulnerable. These exploits could be used for launching DoS attacks or other breaches. Moreover, an attacker could spoof or manipulate control frames (e.g. sending fake EAP-Failure messages or spoofed deauthentication frames) to terminate sessions or fool clients. The researchers concluded that "the current combination of 802.1X and 802.11 does not provide sufficient levels of security, nor will it ever without significant changes"[Arbaugh and Mishra 2002a]. They recommended adding authentication protections to the EAP protocol exchanges and to Wi-Fi management frames, as well as better synchronisation between clients and APs to prevent state desynchronization attacks. This early work highlighted that 802.1X must be implemented with complementary measures to fully achieve secure wireless access control.

Moreover, [Alruban and Everitt 2011] identified two novel DoS attacks targeting the 802.1X/EAP authentication process. The first is an EAP-Nak attack, and the second is an EAP-Notification flooding attack, both of which exploit design features of EAP. In an EAP-Nak-based attack, a malicious supplicant repeatedly responds to the authenticator's EAP requests by sending EAP-Nak packets (which indicate that the requested authentication method is not acceptable), forcing the authenticator to fall back or renegotiate methods. By automating EAP-Nak responses, an attacker can effectively stall or disrupt the normal authentication handshake. The second attack involves abusing EAP-Notification, an optional EAP message type used for displayable messages. In the EAP-Notification flooding attack, an attacker floods the authenticator with a barrage of EAP-Notification messages or prompts. Since EAP-Notification requires processing but is not essential for authentication, this flood can create extra overhead and confusion in the client. Both attacks illustrated that an attacker with the ability to spoof or inject certain EAP packets can exploit the protocol's lack of strict verification for those message types. These Denial-of-Service attacks do not break encryption or steal credentials; they "effectively disrupt the authentication process" [Alruban and Everitt 2011] of 802.1X, making it impossible for legitimate users to connect.

In summary, prior studies show that while 802.1X/EAP greatly improves wireless security, it has protocol-level weaknesses. Unprotected messages can be leveraged by attackers to either hijack sessions or simply deny service to users. These findings set the stage for a practical exploration, where a WPA2-Enterprise network is configured, followed by a demonstration on how an attacker can exploit these weaknesses, specifically focusing on launching a de-authentication flooding attack to repeatedly knock clients offline and force re-authentication.

3. Environment Configuration

To investigate 802.1X vulnerabilities and attacks in practice, a controlled environment implementing a WPA2-Enterprise wireless network was set up. The test environment used a Raspberry Pi running Raspberry Pi OS Lite as the wireless access point and RADIUS server. The built-in wlan0 interface of the Pi was configured in access point mode (via hostapd), and a FreeRADIUS server on the Pi provided 802.1X authentication. Furthermore, dnsmasq was used for the DHCP service on the wireless network, and the Pi's Ethernet (eth0) was used as an uplink (with NAT) to simulate Internet connectivity for clients. Below is a step-by-step description of the configuration:

1. **Install Required Software:** The Raspberry Pi OS Lite system was first updated, and the necessary packages were installed: hostapd (for creating a software access point), FreeRADIUS, and dnsmasq (for DNS/DHCP).

```
sudo apt update && sudo apt upgrade -y
sudo apt install hostapd freeradius dnsmasq -y
```

After installation, these services were stopped temporarily to avoid conflicts during setup.

```
sudo systemctl stop hostapd
sudo systemctl stop dnsmasq
sudo systemctl stop freeradius
```

2. **FreeRADIUS Configuration (Authentication Server):** FreeRADIUS was configured with a simple authentication policy for testing. In the RADIUS users file (/etc/freeradius/3.0/users), a test user account was added with a plaintext password:

```
sudo nano /etc/freeradius/3.0/users
pedro    Cleartext-Password := "senhasecreta"
```

This entry defines a user named “pedro” with a password “senhasecreta”. The RADIUS shared secret (for communication with the authenticator) was set to the same value (“senhasecreta”). After updating the config, FreeRADIUS was launched in debug mode (freeradius -X) to verify that it starts up properly and listens on the default port 1812 for RADIUS authentication.

3. **hostapd Configuration (Wireless Access Point):** Next, the wireless access point and 802.1X authenticator were set up using hostapd. A new hostapd configuration file was created at /etc/hostapd/hostapd.conf with the following content:

```
interface=wlan0
driver=nl80211
ssid=Rede-Lab
hw_mode=g
channel=6
auth_algs=1
ieee8021x=1
eapol_version=2
wpa=2
```

```
wpa_key_mgmt=WPA-EAP
wpa_pairwise=CCMP
rsn_pairwise=CCMP
auth_server_addr=127.0.0.1
auth_server_port=1812
auth_server_shared_secret=senhasecreta
ctrl_interface=/var/run/hostapd
ctrl_interface_group=0
```

This configuration defines an AP on interface wlan0 with SSID “Rede-Lab.” Critically, `ieee8021x=1` and `wpa_key_mgmt=WPA-EAP` enable WPA2-Enterprise (802.1X/EAP authentication). The parameters `auth_server_addr`, `auth_server_port`, and `shared_secret` point to the local RADIUS server (IP 127.0.0.1 and port 1812) with the secret “senhasecreta”(matching what was set in FreeRADIUS and in the hostapd config). We use WPA2 (AES-CCMP) for encryption (`wpa_pairwise=CCMP`), meaning once clients authenticate, they will use dynamically derived AES keys for traffic. After creating this file, the default hostapd service configuration (`/etc/default/hostapd`) was updated to point to it:

```
DAEMON_CONF="/etc/hostapd/hostapd.conf"
```

This ensures the hostapd service will load the config on startup. On Raspberry Pi OS, hostapd is masked by default for manual configuration (it will be unmasked and enabled later).

4. **Network Interface Setup (Static IP and DHCP):** The Raspberry Pi’s wireless interface (wlan0) was configured with a static IP address to serve as the gateway for the wireless LAN. IP 192.168.50.1/24 was chosen for the AP. This was done by editing `/etc/dhcpd.conf` to add:

```
interface wlan0
    static ip_address=192.168.50.1/24
    nohook wpa_supplicant
```

The `nohook wpa_supplicant` line ensures the Pi doesn’t try to act as a client on its own AP interface. Moreover, the IP was applied for the current session with:

```
sudo ip link set wlan0 down
sudo ip addr add 192.168.50.1/24 dev wlan0
sudo ip link set wlan0 up
```

Additionally, dnsmasq was configured for DHCP and DNS on this WLAN. In `/etc/dnsmasq.conf`, the DHCP range and interface were set:

```
interface=wlan0
dhcp-range=192.168.50.10,192.168.50.50,12h
```

The Pi (192.168.50.1) will be the gateway and DNS server for wireless clients by default. Then, IPv4 forwarding was also enabled in `/etc/sysctl.conf` (`net.ipv4.ip_forward=1`) and added a NAT rule so that wireless clients can reach the internet via the Pi’s Ethernet uplink. The NAT was configured by executing:

```
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
sudo netfilter-persistent save
```

which masquerades all traffic from the WLAN (wlan0 subnet) out of the Ethernet interface (eth0). This rule was saved to a file (using iptables-save), and the package iptables-persistent was installed to automatically reapply the NAT setting on boot

5. **Enabling Services:** After completing the configuration, the services were enabled and started in proper order. First, the hostapd was unmasked with `sudo systemctl unmask hostapd`. Then we started FreeRADIUS, dnsmasq, and hostapd, and enabled them to launch at boot:

```
sudo systemctl start freeradius
sudo systemctl enable freeradius
sudo systemctl start dnsmasq
sudo systemctl enable dnsmasq
sudo systemctl start hostapd
sudo systemctl enable hostapd
```

Their status was verified with the command illustrated below to ensure all three were active and running.

```
sudo systemctl status freeradius dnsmasq hostapd
```

At this point, the Raspberry Pi was functioning as a wireless AP named "Rede-Lab" that required 802.1X/EAP authentication. The FreeRADIUS backend accepts the test user pedro/senhasecreta for authentication. Next, a Windows laptop was used as a wireless client configured for WPA2-Enterprise to connect to this network. As shown in Figure 1, the client can connect and reach the internet through the Pi when the correct credentials are provided.

4. EAPOL Handshake Capture

The attacker's first goal is to observe/capture the 802.1X authentication handshake of a client and then disrupt the client's connection by forcing a re-authentication. Capturing the handshake is useful for analysis (and if weak passwords are used, an offline dictionary attack may be possible on the challenge-response). More importantly, forcing repeated handshakes can be used as a denial-of-service tactic.

- **Monitor Mode and Scanning:** The attacker enables monitor mode on their wireless interface to passively capture Wi-Fi frames. Using the airmon-ng tool, any interfering processes were killed (airmon-ng check kill), and our adapter was set to monitor mode on the channel used by the target AP. To capture the 802.1X/EAPOL handshake in Wireshark you need a wireless interface that supports monitor mode. In its default "managed" mode, a Wi-Fi adapter only processes frames addressed to its own MAC, so it will miss the EAPOL packets exchanged between the client and the authenticator. Monitor mode (sometimes called "promiscuous" for wireless) allows the card to tune into a channel and pass every 802.11 frame—including authentication, control and management frames—up to Wireshark. Without a compatible chipset and driver that enable monitor mode, you simply cannot record the full EAPOL 4-way handshake. To make both interfaces compatible, the following commands were used:

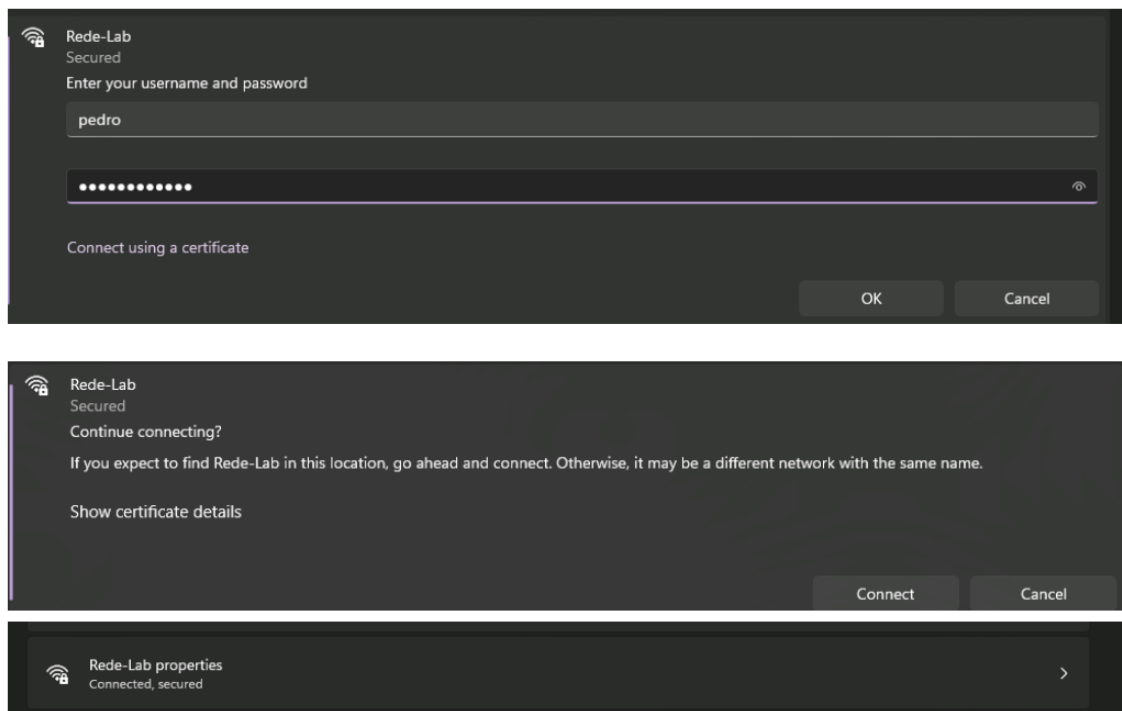


Figure 1. Authentication process to Rede-Lab network.

```
sudo airmon-ng check kill
sudo airmon-ng start wlan0
sudo airdump-ng wlan0
```

In the environment, the network "Rede-Lab" was on channel 6, so our adapter was fixed on channel 6 for capture, as shown in Figure 2. Then, airodump-ng was ran to scan and identify the target AP's BSSID and any connected client devices.

```
sudo iwconfig wlan0 channel 6
```

No.	Time	Source	Destination	Protocol	Length	dst port	info
1006	2025-06-16 01:38:46...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	EAPOL	63		Start
1012	2025-06-16 01:38:46...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	EAP	67		Request, Identity
1083	2025-06-16 01:38:49...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	EAP	67		Request, Identity
1153	2025-06-16 01:38:52...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	EAP	72		Response, Identity
1155	2025-06-16 01:38:52...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	EAP	84		Request, MD5-Challenge EAP (EAP-MD5-CHALLENGE)
1157	2025-06-16 01:38:52...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	EAP	69		Response, Legacy Nak (Response Only)
1162	2025-06-16 01:38:52...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	EAP	68		Response, Protected EAP (EAP-PEAP)
1164	2025-06-16 01:38:52...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	TLSv1.2	239		Ignored Unknown Record
1166	2025-06-16 01:38:52...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	TLSv1.2	165		Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
1167	2025-06-16 01:38:52...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	EAP	165		Response, Protected EAP (EAP-PEAP)
1170	2025-06-16 01:38:52...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	TLSv1.2	119		Change Cipher Spec, Encrypted Handshake Message
1202	2025-06-16 01:38:53...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	EAP	68		Response, Protected EAP (EAP-PEAP)
1204	2025-06-16 01:38:53...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	TLSv1.2	102		Application Data
1206	2025-06-16 01:38:53...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	TLSv1.2	103		Application Data
1208	2025-06-16 01:38:53...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	TLSv1.2	135		Application Data
1210	2025-06-16 01:38:53...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	TLSv1.2	157		Application Data
1212	2025-06-16 01:38:53...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	TLSv1.2	144		Application Data
1215	2025-06-16 01:38:53...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	TLSv1.2	99		Application Data
1217	2025-06-16 01:38:53...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	TLSv1.2	108		Application Data
1219	2025-06-16 01:38:53...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	TLSv1.2	108		Application Data
1220	2025-06-16 01:38:53...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	EAP	66		Success
1224	2025-06-16 01:38:53...	RaspberryPiT_b8:9f:f9	Intel_03:d7:7a	EAPOL	213		Key (Message 3 of 4)
1226	2025-06-16 01:38:53...	Intel_03:d7:7a	RaspberryPiT_b8:9f:f9	EAPOL	157		Key (Message 4 of 4)

Figure 2. Wireshark packet capture screenshot.

- **Packet Capture:** In the Wireshark capture, illustrated in Figure 3, you can see the full 802.1X/EAPOL exchange followed by the WPA 4-way key handshake. It begins with frame 1006 (EAPOL Start) from the client (Intel_03:d7:7a) to the authenticator (RaspberryPiT_b8:9f:f9), triggering two EAP-Request Identity messages. Next, the client's EAP-Response Identity prompts the server to offer an MD5-Challenge, but the client replies with a "Legacy Nak", indicating it will not use MD5. Negotiation then shifts to PEAP: you see the protected EAP-PEAP initiation, followed by the TLSv1.2 handshake embedded in EAP, including Client Key Exchange, Change Cipher Spec, and Encrypted Handshake messages. Once the PEAP tunnel is established, multiple "Application Data" records flow over TLS, concluding with the EAP Success message. Immediately afterwards, the WPA 4-way handshake executes over EAPOL. This sequence demonstrates how 802.1X negotiates a secure tunnel before deriving WLAN encryption keys.

5. Flooding Attack Demonstration (Deauth DoS)

An attack on a wireless 802.1X network is a deauthentication flooding attack, which is a form of denial-of-service where the attacker continuously transmits forged deauth frames, forcing clients to disconnect and re-authenticate over and over. Because the 802.1X/EAP process is time-consuming, a constant flood of deauth packets can effectively make the network unusable, as clients never get a chance to finish authentication or send any useful data before being kicked off again. Such attack was performed on the "Rede-Lab" network using the aireplay-ng tool in a mode that sends infinite deauthentication frames.

First, the attacking wireless interface was put on the same channel as the target AP (channel 6). Then the following command was ran:

```
sudo aireplay-ng --deauth 0 -a DC:A6:32:B8:9F:F9 wlan1 /  
--ignore-negative-one
```

Here, `--deauth 0` means "send deauth frames indefinitely" (0 is interpreted as no limit, i.e., keep sending until stopped). The `-a DC:A6:32:B8:9F:F9` specifies the target AP's MAC address to spoof as the source; no particular client was specified (`-c`), which means that the deauth frames were sent to the broadcast address, targeting all clients on that network. The `--ignore-negative-one` flag was used to bypass a driver warning. As soon as the command was executed, the attacking adapter began injecting a continuous stream of de-authentication frames.

Moreover, as seen in Figure 4, aireplay-ng reports it is sending DeAuth frames to the broadcast address on behalf of the AP. Wireshark was opened on the attacker machine to verify the attack traffic; a rapid stream of deauth packets was observed being sent out every second.

The effect on the victim client was immediate: the Windows laptop that was connected to "Rede-Lab" started getting disconnected repeatedly. Essentially, the client would receive a deauth, drop the connection, and then automatically try to reconnect. After a short while, the network was effectively unusable for that client. In a real-world scenario, this would manifest as the Wi-Fi network being up but nobody being able to maintain a connection.

No.	Time	Source	Destination	Protocol	Length	Info
3497	35.621326859	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=0, FN=0, Flags=.....
3501	35.640880209	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=1, FN=0, Flags=.....
3502	35.641062881	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=0, FN=0, Flags=.....
3503	35.641514392	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=1, FN=0, Flags=.....
3504	35.643113260	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=2, FN=0, Flags=.....
3506	35.645233897	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=2, FN=0, Flags=.....
3507	35.645390786	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=3, FN=0, Flags=.....
3508	35.646207673	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=3, FN=0, Flags=.....
3509	35.647571252	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=4, FN=0, Flags=.....
3510	35.649690509	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=4, FN=0, Flags=.....
3511	35.649841541	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=5, FN=0, Flags=.....
3512	35.664909978	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=5, FN=0, Flags=.....
3518	35.764820796	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=6, FN=0, Flags=.....
3520	35.772137366	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=6, FN=0, Flags=.....
3521	35.774825835	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=7, FN=0, Flags=.....
3522	35.779490053	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=7, FN=0, Flags=.....
3523	35.780378683	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=8, FN=0, Flags=.....
3524	35.781223714	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=8, FN=0, Flags=.....
3525	35.783263552	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=9, FN=0, Flags=.....
3526	35.784599069	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=9, FN=0, Flags=.....
3527	35.786346085	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=10, FN=0, Flags=.....
3528	35.787550284	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=10, FN=0, Flags=.....
3529	35.790279650	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=11, FN=0, Flags=.....
3530	35.794035623	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=11, FN=0, Flags=.....
3531	35.794571099	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=12, FN=0, Flags=.....
3532	35.796276141	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=12, FN=0, Flags=.....
3533	35.796648072	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=13, FN=0, Flags=.....
3534	35.801970308	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=13, FN=0, Flags=.....
3535	35.802415066	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=14, FN=0, Flags=.....
3536	35.803534863	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=14, FN=0, Flags=.....
3537	35.808115852	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=15, FN=0, Flags=.....
3538	35.811446430	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=15, FN=0, Flags=.....
3539	35.812682016	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=16, FN=0, Flags=.....
3540	35.812685014	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=16, FN=0, Flags=.....
3542	35.820808354	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=17, FN=0, Flags=.....
3544	35.835809247	RaspberryPiT_b8:9f:...	Broadcast	802.11	37	Deauthentication, SN=17, FN=0, Flags=.....
3545	35.836414110	RaspberryPiT_b8:9f:...	Broadcast	802.11	38	Deauthentication, SN=18, FN=0, Flags=.....

Figure 3. Wireshark packet capture output of the de-authentication flooding attack.

Moreover, death floods can also be used in combination with evil twin attacks, for example, an attacker kicks everyone off a legitimate network to entice them to connect to a rogue AP. In this article, we focused on pure denial-of-service: keeping the authorised users off their network. Additionally, the flood can be scaled, the flood can be client-specific, so a specific device is kicked off the network (`-c <MAC>`), or use broadcast (as illustrated in the attack) to take down the entire network segment. The attack works against any 802.11 network (not just 802.1X) that doesn't have management frame protection.

6. Mitigations for Flooding Attacks in 802.1X

Denial-of-service attacks targeting the 802.1X authentication process (such as EAPOL flooding, deauthentication flooding, or EAP message abuse) can be countered by layered technical controls. In the following subsection, four mitigation strategies will be explored.

6.1. Protected Management Frames

The IEEE 802.11w amendment provides cryptographic protection for critical management frames, including deauthentication and disassociation. With 802.11w enabled, clients and APs will reject forged deauthentication frames that are not properly encrypted/signed, thereby mitigating the classic death flood attack. This measure effectively “hardens” the 802.1X WLAN against management frame spoofing – an attacker’s fake disconnection frames will be ignored by stations that support Protected Management Frames. Enabling 802.11w on an Enterprise WLAN is therefore a primary defence against de-authentication flooding, ensuring that only management frames with valid cryptographic protection can influence client connectivity.

6.2. EAP Rate Limiting

Rate limiting the frequency of EAP authentication attempts can mitigate floods of bogus authentication traffic. Attackers who script repeated EAPOL-Start or fake supplicant requests can overwhelm an AP or RADIUS server; to counter this, the authenticator should enforce thresholds on authentication dialogues. For example, temporarily blocking a client for a cooldown period after a small number of failed or rapid-fire EAP attempts is an effective strategy. Such throttling prevents a malicious supplicant from continuously hammering the 802.1X exchange. It also serves as a safeguard against brute-force credential guessing by limiting how quickly an attacker can iterate through attempts. Implementing these limits at the AP or switch (authenticator) ensures that excessive EAP requests never reach the RADIUS server at full rate, preserving backend availability. In essence, well-tuned EAP rate limiting forces an attacker into a timeout after a few rapid attempts, reducing the impact of EAP flooding attacks.

6.3. Wireless Intrusion Detection/Prevention Systems

Many enterprise WLAN infrastructures include IDS/IPS capabilities to monitor for anomalies and attack signatures in real time. These systems can detect patterns characteristic of flooding attacks. For instance, a high volume of de-authentication frames or an unusual high in EAPOL traffic, and automatically take action. At minimum, an alert is generated to network administrators, but often the response can be configured to actively mitigate (e.g. temporarily block the attacker's MAC address or channel). WIPS can also monitor for EAP-based abuses, such as an abnormal sequence of EAP-Failure messages or repeated EAP negotiation restarts, and flag these as attack indicators. By deploying wireless IDS/IPS tools or enabling built-in AP security monitoring, administrators add an extra layer of defence that can react to flooding attacks in progress.

6.4. RADIUS Server Throttling

In addition, the RADIUS authentication server can be configured to withstand bursts of authentication attempts. Countermeasures at the server level include configuring request rate limits and adding logic to ignore or delay repeated failures from the same source. For example, if a particular client's MAC or username fails authentication multiple times in quick succession, the RADIUS server can deliberately slow the response or temporarily refuse further attempts from that client. This kind of throttling ensures that a flood of EAP requests does not bring the authentication server to a halt. Ensuring the RADIUS service is properly scaled for the client load and employing features like challenge-response (which forces an attacker to perform extra work per attempt) can work toward mitigating a flooding attack.

7. Conclusion

In this article, we demonstrated how a Raspberry Pi-based WPA2-Enterprise environment combining FreeRADIUS, hostapd, and dnsmasq replicates an enterprise 802.1X deployment, complete with EAPOL handshake and dynamic key generation. Moreover, despite its strong authentication framework, 802.1X remains vulnerable since management and de-authentication frames lack integrity protection, allowing an attacker to launch a persistent deauth flood. This attack prevents clients from ever completing the EAP exchange

or installing encryption keys, effectively knocking the network offline without touching its cryptographic algorithms.

Our packet captures and Wireshark analysis made clear how easily an adversary can observe the entire EAPOL sequence: identity requests, PEAP tunnel negotiation, TLS exchanges, and the 4-way key handshake, and how a few forged deauth frames undo all that work in real time. To counter such denial-of-service tactics, network operators should enable 802.11w Protected Management Frames, impose rate limits on EAP messages, and deploy wireless intrusion detection systems that flag abnormal deauth traffic. On the RADIUS side, configuring blacklists or throttling repeated authentication failures can blunt EAP-based floods. In conclusion, securing a wireless LAN requires a layered defence that not only authenticates users and encrypts data but also ensures availability by protecting the very messages that keep devices connected.

References

- Alruban, A. and Everitt, E. (2011). Two Novel 802.1x Denial of Service Attacks. In *Proceedings of the IEEE International Symposium on Policies for Distributed Systems and Networks (POLICY)*, pages 178–185.
- Arbaugh, W. A. and Mishra, A. (2002a). An Initial Security Analysis of the IEEE 802.1X Standard. Technical Report CSI-TR-2002-2004, University of Maryland, College Park.
- Arbaugh, W. A. and Mishra, A. (2002b). An Initial Security Analysis of the IEEE 802.1X Standard. Technical Report ADA422412, Defense Technical Information Center.
- Armitage, S. (2011). Known Wireless Attacks. Jisc Community Advisory Services: Wireless Technology Advisory Service. Accessed: 2025-06-20.
- Cisco Systems, Inc. (n.d.). Broadcast deauthentication frame signatures. Cisco Wireless Controller Configuration Guide, Release 7.6, Chapter “Configuring IDS Signatures”. Accessed: 2025-06-20.
- Debian Wiki (2025). Dnsmasq. [wiki.debian.org](https://wiki.debian.org/Dnsmasq). Accessed June 2025.
- Fortinet (2025). 802.1X Authentication. Fortinet Cyber Glossary. Accessed June 2025.
- Gentoo Wiki (2025). Hostapd. [wiki.gentoo.org](https://wiki.gentoo.org/wiki/Hostapd). Accessed June 2025.
- Ixmpove (2020). Deauthentication Attack Using aireplay-ng. Dev.to. Accessed June 2025.
- NetworkRADIUS (2025). FreeRADIUS Documentation. networkradius.com. Accessed June 2025.
- Notter, R. (2018). 802.11w-2009 – Protected Management Frames (PMF): Overview and Lab Tests. Accessed: 2025-06-20.
- Nzyme Project (2025). Deauthentication Frames Explained. nzyme.org. Accessed June 2025.
- SecureW2 (2025). 802-1X Solutions and Use Cases. securew2.com. Accessed June 2025.
- The Register (2002). 802.1X can be toppled ‘like a set of dominoes’. *The Register*. Accessed June 2025.